

Лабораторна робота №2. RAG-система над відкритим корпусом

25 серпня 2026 р.

Мета

Побудувати пошукову систему над реальним українським корпусом, довести числами, що дає кожна зміна конфігурації, і зробити так, щоб відповідь без джерела була неможливою.

Робота виконується **бригадою з трьох осіб**. Лекції 6–8.

Усе безкоштовне. Корпус береться з відкритих даних, моделі embeddings — з Hugging Face, індекси локальні. Платних сервісів не потрібно.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — корпус і ingestion.** Збирає корпус із відкритих джерел із перевіреними ліцензіями, робить парсинг, чанкінг і метадані, відповідає за те, щоб текст у чанках був придатним для пошуку.
- **Учасник В — індекси і пошук.** Будує розріджений і щільний індекси, гібрид через RRF, підключає кросенкодерний реранкер, відповідає за конфігурації та затримки.
- **Учасник С — розмітка і оцінювання.** Складає розмічений набір запитів, реалізує метрики й парний bootstrap, будує генерацію з цитуванням і відмовою.

Інтеграційний крок виконують разом: чотири конфігурації учасника В проганяються на корпусі учасника А і оцінюються набором учасника С. Ablation планує вся бригада, а виконує той, чию підсистему він зачіпає.

Теоретичні відомості

Retrieval-augmented generation з'єднує пошук по зовнішньому сховищу з генерацією: модель отримує знайдені фрагменти в контексті і будує відповідь із них, а не з ваг. Оцінювати таку систему як одне ціле марно, бо ламаються дві різні речі: пошук може не знайти потрібний фрагмент, або знайти, а модель його проігнорує.

- **Одиницею пошуку є фрагмент, а не документ.** Разом із текстом зберігаються метадані: джерело, розділ, дата, права доступу. Без них неможливі ні фільтрація, ні цитування, ні видалення застарілого. Факт, розрізаний межею чанка, не знайде жоден пошук — саме тому перекриття не косметичне.
- **Розріджений пошук зіставляє слова:**

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 (1 - b + b \frac{|d|}{\text{avgdl}})}$$

Він сильний на точних термінах, номерах наказів і назвах, і слабкий на перефразуванні. Щільний пошук зіставляє змісти й помиляється навпаки — тому їх об'єднують.

- **Reciprocal Rank Fusion працює з рангами, а не зі шкалами,** тому не потребує їх узгодження:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60$$

- **Реранкер ставлять другим етапом.** Бієнкодер кодує запит і документ окремо, тому вектори рахуються заздалегідь; кросенкодер подає пару разом і тому точніший, але застосовний лише до кількох десятків кандидатів.
- **Порівнюють не два окремі інтервали, а інтервал різниці.** Дві конфігурації на тому самому наборі дають парні спостереження: для кожного запиту береться різниця Recall@5, і 95 % інтервал будується вже для середньої різниці — парним bootstrap. Висновок «інтервали перекриваються, отже різниці немає» некоректний. Кластери враховують окремо: десять запитів до одного документа — це одне спостереження з десятьма гранями.
- **Контракт відповіді важливіший за красу тексту.** Кожне твердження супроводжується вказівником на чанк, а за відсутності релевантного контексту система відмовляється відповідати. Дозвіл промовчати задається текстом промпта так само явно, як і решта інструкції.

Корисні посилання для ознайомлення:

- [Відкриті дані України](#) і [датасети Hugging Face](#) як джерела корпусу;
- Lewis P. та ін. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#), NeurIPS 2020;
- [sqlite-vec](#), [FAISS: як обрати індекс](#), [Chroma](#);
- [MTEB Leaderboard](#) — порівняння моделей embeddings за мовами;
- Anthropic, [Introducing Contextual Retrieval](#).

Порядок виконання

Позначка [A], [B], [C] вказує відповідального за крок; **[разом]** — крок виконує вся бригада.

1. **[разом] Обрати предметну область** і сформулювати десять типових питань, на які система має відповідати. Ці питання пізніше стануть частиною розміченого набору і не змінюються.
2. **[A] Зібрати корпус:** 50–150 відкритих документів однієї області з явно зазначеною ліцензією. Джерела і ліцензії звести в таблицю. Документ без зрозумілої ліцензії до корпусу не потрапляє.
3. **[A] Реалізувати ingestion:** парсинг, очищення, чанкінг із перекриттям, метадані (джерело, сторінка або розділ, дата). Обґрунтувати розмір чанка, порахувати кількість чанків і розмір індексу.
Перевірка, яка коштує хвилину і рятує день: узяти точну фразу з паперового документа і пошукати серед чанків. Не знайшли — далі йти немає сенсу.
4. **[C] Скласти розмічений набір:** не менш ніж 60 запитів природною мовою, для кожного вручну позначені правильні чанки; серед них десять питань із кроку 1 і не менше п'яти свідомо поза корпусом. Набір фіксується до експериментів.
5. **[B] Побудувати два індекси:** розріджений (`rank_bm25`) і щільний (`sentence-transformers` + `sqlite-vec` або `FAISS`).
Дотриматися контракту моделі: для сімейства E5 обов'язкові префікси `query:` і `passage::`; для косинуса у `FAISS` — нормалізація разом із `METRIC_INNER_PRODUCT`.
6. **[B] Зібрати чотири конфігурації:** лише BM25; лише щільний; гібрид через RRF; гібрид із кросенкодерним реранкером `top-20` → `top-5`. Для кожної зняти `Recall@5`, `MRR` і затримку `p50`.

7. **[C] Порахувати інтервали різниці:** для кожної пари конфігурацій — 95 % інтервал парної різниці per-query Recall@5, парний bootstrap із ресемплом за документами.
Головний результат роботи: назвати пари, різниця яких накриває нуль. Відповідь «різниці немає» є повноцінним результатом.
8. **[C] Реалізувати генерацію з цитуванням:** кожне твердження супроводжується вказівником на чанк; за відсутності контексту система відмовляється. Перевірити на 15 запитах, серед яких п'ять поза корпусом.
9. **[разом] Спланувати і провести ablation:** одна зміна за вибором бригади — розмір чанка, перекриття, кількість кандидатів, модель embeddings або efSearch. Змінюється рівно один параметр, набір лишається тим самим.
10. **[разом] Побудувати демонстрацію:** найпростіший інтерфейс — командний рядок або сторінка — де видно питання, знайдені чанки з їхніми оцінками і відповідь із цитатами.
11. **[разом] Оформити ноутбук.** Зібрати роботу в один Jupyter-ноутбук: код кожного учасника у своїх комірках із короткими коментарями. Обов'язково всередині: таблиця джерел і ліцензій, характеристики корпусу після ingestion, таблиця чотирьох конфігурацій з інтервалами парних різниць, результат ablation, п'ятнадцять прикладів відповідей із цитатами і висновок до 250 слів про те, яка зміна дала найбільший приріст на одиницю затримки.

Порядок кроків 4 і 5 не переставляти: набір, складений після того, як побачили видачу індексу, міряє індекс, а не задачу.

Контрольні запитання

1. Чому оцінювання retriever і generator розділяють?
2. Що станеться з BM25, якщо параметр b дорівнює нулю?
3. Чому косинус між векторами двох різних моделей не має сенсу?
4. Навіщо в RRF константа 60 і що буде, якщо взяти 1?
5. Чому кросенкодер не застосовують до всієї бази?
6. Recall@5 зріс, а якість відповідей — ні. Які причини можливі?
7. Чому перекриття двох маргінальних інтервалів не є доказом відсутності різниці?
8. Чому ресемпл у bootstrap робиться за документами, а не за запитами?

9. Чому набір фіксується до експериментів?
10. Коли пошук взагалі не потрібен і документи можна подати в контекст цілком?

Література

1. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. *FnTIR*, 2009.
2. Malkov Yu., Yashunin D. [Efficient and Robust ANN Search Using HNSW Graphs](#). *TPAMI*, 2018.
3. Thakur N. та ін. [BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of IR Models](#). *NeurIPS*, 2021.
4. Gao Y. та ін. [Retrieval-Augmented Generation for LLMs: A Survey](#), 2024.
5. Miller E. [Adding Error Bars to Evals](#). *arXiv:2411.00640*, 2024.